Efficient Methods for Solving the Traveling Salesman Problem

Filippo Tiberio Herman Serain

March 2026

Abstract

The Traveling Salesman Problem (TSP) is one of the most well-known and widely studied problems in combinatorial optimization. Despite its simple formulation, the problem quickly becomes computationally intractable as the number of cities increases. Because of its theoretical importance and its wide range of practical applications, the TSP has been the subject of extensive research for several decades.

This report introduces the Traveling Salesman Problem and discusses the main techniques used to solve it efficiently. After presenting the formal definition of the problem and its mathematical representation, we briefly review the computational challenges associated with it. The goal of this work is to provide an overview of the main algorithmic approaches used to address the TSP, highlighting the trade-offs between optimality and computational efficiency. In particular, we examine construction and refinement heuristics, metaheuristic strategies, and exact methods based on integer linear programming.

Contents

1 Introduction **3**

2 Background Material **3**
2.1 What is the Traveling Salesman Problem . . . 3
2.2 IBM CPLEX Optimizer . . . 4
2.3 Performance Profiles . . . 4

3 Heuristic Methods **5**
3.1 Construction Heuristics . . . 5
3.1.1 Nearest Neighbor Heuristic . . . 5
3.1.2 Extra Mileage Heuristic . . . 6
3.2 Refinement Heuristics . . . 7
3.2.1 2-opt Refinement . . . 7

4 Metaheuristic Methods **8**
4.1 Greedy Randomized Adaptive Search Procedure (GRASP) . . . 8
4.1.1 Hyperparameter Tuning for GRASP . . . 9
4.2 Variable Neighborhood Search (VNS) . . . 10
4.2.1 Hyperparameter Tuning for VNS . . . 11
4.3 Tabu Search . . . 12
4.3.1 Hyperparameter Tuning for Tabu Search . . . 13
4.4 Comparison of Heuristic and Metaheuristic Methods . . . 14

5 Exact Methods **15**
5.1 Integer Linear Programming Formulation . . . 15
5.2 Implementation with CPLEX . . . 16
5.2.1 Model Construction (`build_model`) . . . 16
5.2.2 Solving the Model (`TSPopt`) . . . 16
5.2.3 Tour Reconstruction (`build_sol`) . . . 16
5.3 Comparison with Heuristic Methods . . . 17

6 Conclusions **17**

1 Introduction

The Traveling Salesman Problem (TSP) is a classical problem in combinatorial optimization and theoretical computer science. Informally, the problem can be described as follows: given a set of cities and the distances between every pair of them, the objective is to determine the shortest possible route that visits each city exactly once and returns to the starting city.

Although the problem is easy to state and understand, finding the optimal solution is computationally challenging. The difficulty arises from the extremely large number of possible tours that must be considered. For a problem with n cities, the number of potential routes grows factorially with n. Even for relatively small instances, the number of possible solutions becomes prohibitively large, making exhaustive search impractical.

The TSP is classified as an NP-hard problem in computational complexity theory. This means that no algorithm is currently known that can solve all instances of the problem in polynomial time. As a consequence, a large portion of research has focused on developing more efficient algorithms and approximation techniques that can produce optimal or near-optimal solutions within a reasonable computational time.

Beyond its theoretical importance, the Traveling Salesman Problem has numerous practical applications. It appears naturally in many real-world scenarios where a set of locations must be visited in the most efficient order. Examples include logistics and delivery route planning, vehicle routing problems, manufacturing processes such as printed circuit board drilling, and even certain problems in computational biology.

Because of its importance, the TSP has become a benchmark problem used to test and compare optimization algorithms. Over the years, researchers have developed a wide variety of solution approaches, including exact algorithms, approximation algorithms, and heuristic methods. Each of these approaches presents different advantages in terms of accuracy, computational cost, and scalability.

In this report, we introduce the fundamental concepts behind the Traveling Salesman Problem and discuss the main approaches used to solve it efficiently. We begin by presenting the formal definition of the problem and its mathematical representation, which will serve as the foundation for the methods discussed later.

2 Background Material

2.1 What is the Traveling Salesman Problem

The Traveling Salesman Problem is a combinatorial optimization problem defined on a set of nodes representing cities and edges representing the distances between them. The objective is to determine the shortest closed tour that visits every city exactly once and returns to the starting point.

From a mathematical perspective, the problem is typically modeled using a weighted graph. Let $G = (V, E)$ be a graph where V is the set of vertices representing the cities and E is the set of edges connecting pairs of cities. Each edge (i, j) is associated with a cost or distance d_{ij} representing the travel cost between city i and city j.

The goal is to find a tour that minimizes the total travel distance while ensuring that each city is visited exactly once. This can be expressed as the minimization of the total cost of the tour:

$$\min \sum_{i=1}^{n} d_{\pi(i),\pi(i+1)}$$

where π represents a permutation of the vertices and $\pi(n+1) = \pi(1)$ ensures that the route returns to the starting city.

The TSP can appear in several different variants depending on the properties of the distance matrix. One of the most common forms is the *symmetric Traveling Salesman Problem*, in which the distance between two cities is the same in both directions:

$$d_{ij} = d_{ji}$$

In contrast, the *asymmetric Traveling Salesman Problem* allows the distance from city i to city j to differ from the distance from city j to city i. This situation may arise in real-world applications such as road networks with one-way streets or different travel costs depending on direction.

A key difficulty of the TSP lies in the rapid growth of the solution space. For a symmetric TSP with n cities, the number of possible tours is approximately

$$\frac{(n-1)!}{2}$$

since many tours are equivalent due to symmetry and rotation. Even for moderately sized problems, the number of possible tours becomes extremely large. For example, with only 20 cities there are already more than 10^{18} possible tours.

Because of this combinatorial explosion, solving the TSP efficiently requires sophisticated optimization techniques. Over the years, researchers have developed a wide range of approaches, including exact algorithms capable of guaranteeing optimal solutions, as well as heuristic and metaheuristic methods that can find high-quality solutions for very large instances.

Understanding the structure of the TSP and the challenges associated with it is essential before exploring these solution methods. In the following sections, we will discuss some of the most important algorithmic strategies developed to tackle this problem.

2.2 IBM CPLEX Optimizer

IBM CPLEX Optimizer, hereafter referred to simply as CPLEX, is a high-performance mathematical programming solver developed and maintained by IBM. It is widely used in operations research and combinatorial optimization to solve a broad class of problems, including linear programs (LP), mixed-integer programs (MIP), and quadratic programs (QP). For the purposes of this work, CPLEX is employed to solve the TSP formulated as an integer linear program (ILP), providing provably optimal solutions for moderate-sized instances.

CPLEX exposes a rich C API, through which the user can programmatically build a model by adding variables, objective functions, and constraints, and then invoke the solver. The key API objects are the `CPXENVptr` environment handle, which manages solver parameters and licensing, and the `CPXLPptr` problem handle, which stores the mathematical model.

For this project, CPLEX version 22.1 was used on a Linux x86-64 machine, a WIN11 machine and a macOS Tahoe (version 26.5) machine. The solver is invoked exclusively through its C callable library.

2.3 Performance Profiles

When comparing multiple optimization algorithms on a set of benchmark instances, a common challenge is that no single scalar metric captures the full picture of an algorithm's behavior. An algorithm may be superior on some instances and inferior on others; moreover, the degree of superiority matters. The *performance profile*, introduced by Dolan and Moré [1], is a widely adopted tool in the operations research community for conducting such comparisons in a statistically robust and visually informative manner.

Definition. Let $\mathcal{S}$ be the set of solvers (algorithms) to be compared and $\mathcal{P}$ be the set of benchmark problem instances. For each solver $s \in \mathcal{S}$ and each instance $p \in \mathcal{P}$, let $t_{p,s}$ denote the performance metric of interest (e.g., solution cost or CPU time). The *performance ratio* of solver s on instance p is defined as:

$$r_{p,s} = \frac{t_{p,s}}{\min_{s' \in \mathcal{S}} t_{p,s'}}$$

This ratio measures how far solver s is from the best solver on instance p. A ratio of 1 means the solver achieved the best result on that instance. The performance profile of solver s is then the cumulative distribution function of the performance ratios:

$$\rho_s(\tau) = \frac{1}{|\mathcal{P}|} \left|\{p \in \mathcal{P} : r_{p,s} \leq \tau\}\right|$$

where $\tau \geq 1$ is the performance ratio threshold. Intuitively, $\rho_s(\tau)$ gives the fraction of instances on which solver s is within a factor τ of the best solver.

Interpretation. The value $\rho_s(1)$ represents the fraction of instances on which solver s achieves the best performance among all solvers — a measure of *winning frequency*. As τ increases, $\rho_s(\tau)$ approaches 1 (the solver eventually becomes competitive on all instances). An algorithm with a profile that lies above and to the left of all others is uniformly superior. When profiles cross, the interpretation depends on the specific operating region of τ that is of practical interest.

In this work, performance profiles are used to compare the solution quality achieved by the various heuristic, metaheuristic and math-heuristic algorithms described in the following sections, evaluated over a suite of randomly generated TSP benchmark instances. The metric adopted is the normalized solution cost, defined as the ratio of the cost obtained by each method to the best known solution cost for each instance. This normalization makes performance profiles scale-independent and suitable for aggregating results across instances of different sizes.

3 Heuristic Methods

Heuristic methods do not guarantee optimality but are designed to find good-quality solutions within a reasonable amount of time. They are particularly valuable for large TSP instances where exact methods become computationally prohibitive. In this work, heuristics are organized into two categories: *construction heuristics*, which build a complete tour from scratch, and *refinement heuristics*, which iteratively improve an existing tour.

3.1 Construction Heuristics

Construction heuristics produce an initial feasible solution by incrementally assembling a tour. While the resulting solutions are generally not optimal, they serve as high-quality starting points for subsequent refinement procedures.

3.1.1 Nearest Neighbor Heuristic

The Nearest Neighbor (NN) heuristic is one of the simplest and most widely known greedy construction methods for the TSP. Its appeal lies in its conceptual simplicity and low computational overhead. The algorithm builds a tour by iteratively extending the current partial tour with the unvisited city that is closest to the last city added.

Algorithm Description. Starting from an initial city, the heuristic greedily selects at each step the nearest unvisited city and appends it to the current tour. Once all cities have been visited, the tour is closed by returning to the starting city. The procedure is repeated from every city as a starting node, and the best tour found across all starting points is retained as the final solution.

The pseudocode for a single iteration of the Nearest Neighbor heuristic is given in Algorithm 1.

Algorithm 1 Nearest Neighbor Heuristic (single starting node)

Require: Instance with n cities, starting node s
Ensure: Tour π and its cost C
1: $\pi \leftarrow [0, 1, \ldots, n{-}1]$
2: **swap** $\pi[0] \leftrightarrow \pi[s]$
3: $C \leftarrow 0$
4: **for** $i = 0$ **to** $n - 3$ **do**
5: best $\leftarrow \arg\min_{j \in \{i+1,\ldots,n-1\}} d_{\pi[i],\,\pi[j]}$
6: **swap** $\pi[i{+}1] \leftrightarrow \pi[\text{best}]$
7: $C \leftarrow C + d_{\pi[i],\,\pi[i+1]}$
8: **end for**
9: $C \leftarrow C + d_{\pi[n-2],\,\pi[n-1]} + d_{\pi[n-1],\,\pi[0]}$ ▷ Last edge + closing edge
10: **return** π, C

Implementation Details. In our implementation, the distances between all pairs of cities are precomputed and stored in a compact upper triangular matrix to avoid redundant calculations during the heuristic loop. Access to the precomputed distance between nodes i and j (with $i < j$) is realized through the index formula:

$$\text{index}(i, j, N) = i \cdot N - \frac{i(i+1)}{2} + (j - i - 1)$$

which maps the upper triangle of the distance matrix to a one-dimensional array of size $\binom{N}{2}$. The multi-start strategy, running the heuristic from each city as a starting point, is implemented with a randomized ordering of starting nodes to allow early termination within a given time budget while still exploring diverse starting points.

Complexity. Each iteration of the Nearest Neighbor heuristic from a fixed starting node requires $O(n^2)$ time in the basic implementation, as for each of the n cities to be placed, all remaining unvisited cities must be examined. Running the multi-start variant over all n starting nodes results in an overall complexity of $O(n^3)$.

3.1.2 Extra Mileage Heuristic

The Extra Mileage (EM) heuristic, also known as the cheapest insertion heuristic, builds a tour by starting with a minimal sub-tour and incrementally inserting the remaining cities at the position that incurs the least additional cost. Compared to the Nearest Neighbor approach, it tends to produce more balanced tours because it considers the impact of each insertion on the existing partial tour.

Algorithm Description. The heuristic begins by identifying the two most distant cities, which form the initial sub-tour of length 2 (a back-and-forth path between those two cities). At each subsequent step, for every unvisited city k and every edge (i, j) in the current partial tour, the *extra mileage* required to insert k between i and j is computed as:

$$\Delta(k, i, j) = d_{ik} + d_{kj} - d_{ij}$$

The city k^* and insertion position (i^*, j^*) that minimize Δ are selected, and k^* is inserted between i^* and j^* in the tour. This process repeats until all cities have been inserted.

Algorithm 2 Extra Mileage Heuristic

Require: Instance with n cities
Ensure: Tour π and its cost C

$(u, v) \leftarrow \arg\max_{i<j}\ d_{ij}$
$\pi \leftarrow [u, v]$, visited$[u] \leftarrow$ visited$[v] \leftarrow$ true, $m \leftarrow 2$
while $m < n$ **do**
    $\Delta^* \leftarrow +\infty$, $k^* \leftarrow -1$, $p^* \leftarrow -1$
    for each unvisited city k **do**
        for $i = 0$ **to** $m - 1$ **do**
            $j \leftarrow (i + 1) \bmod m$
            $\Delta \leftarrow d_{\pi[i],\,k} + d_{k,\,\pi[j]} - d_{\pi[i],\,\pi[j]}$
            if $\Delta < \Delta^*$ **then**
                $\Delta^* \leftarrow \Delta$, $k^* \leftarrow k$, $p^* \leftarrow i + 1$
            end if
        end for
    end for
    Shift right $\pi[p^* \,..\, m]$ by one position ▷ Make room
    $\pi[p^*] \leftarrow k^*$, visited$[k^*] \leftarrow$ true, $m \leftarrow m + 1$
end while
$C \leftarrow$ TourCost(π)
return π, C

Complexity. The Extra Mileage heuristic has a worst-case time complexity of $O(n^3)$, since at each of the $n-2$ insertion steps, it must evaluate up to n unvisited cities against up to n edges in the current partial tour. In practice, the constant factors are slightly larger than those of the Nearest Neighbor heuristic, making EM somewhat slower for very large instances.

3.2 Refinement Heuristics

Refinement heuristics take an existing feasible tour and attempt to improve it by performing local modifications. They are typically applied as a post-processing step after a construction heuristic has produced an initial solution, and they continue iterating until no further improvement is possible or a time budget is exhausted.

3.2.1 2-opt Refinement

The 2-opt algorithm, originally proposed by Lin (1965) [2], is the most widely used local search refinement method for the TSP. It works by repeatedly selecting two edges in the current tour and replacing them with two alternative edges, effectively reversing the segment of the tour between the two selected edges. This operation is called a *2-opt move*.

2-opt Move. Given a tour $\pi = [\pi_1, \pi_2, \ldots, \pi_n]$, a 2-opt move selects two non-adjacent edges (π_i, π_{i+1}) and (π_j, π_{j+1}) and replaces them with the edges (π_i, π_j) and (π_{i+1}, π_{j+1}). The segment $[\pi_{i+1}, \ldots, \pi_j]$ is then reversed, yielding a new tour. The cost change of such a move is:

$$\Delta C = \big(d_{\pi_i,\pi_j} + d_{\pi_{i+1},\pi_{j+1}}\big) - \big(d_{\pi_i,\pi_{i+1}} + d_{\pi_j,\pi_{j+1}}\big)$$

A move is accepted if and only if $\Delta C < 0$, i.e., the new tour is strictly shorter.

Best-Improvement Strategy. Our implementation uses the *best-improvement* strategy: in each iteration, all $O(n^2)$ possible 2-opt moves are evaluated, and the one with the largest improvement (ΔC most negative) is applied. This contrasts with the *first-improvement* strategy, which applies the first improving move found. While best-improvement typically requires more evaluations per iteration, it often converges to a better local optimum.

Algorithm 3 2-opt Refinement

Require: Tour π of n cities, cost C
Ensure: Locally optimal tour π, cost C

repeat
  $\Delta^* \leftarrow 0$, $a^* \leftarrow -1$, $b^* \leftarrow -1$
  for $i \leftarrow 0$ **to** $n-2$ **do**
    for $j \leftarrow i+2$ **to** $n-1$, $j \neq (i+n-1) \bmod n$ **do**
      $i' \leftarrow (i+1) \bmod n, \quad j' \leftarrow (j+1) \bmod n$
      $\Delta C \leftarrow \big(d_{\pi[i],\pi[j]} + d_{\pi[i'],\pi[j']}\big) - \big(d_{\pi[i],\pi[i']} + d_{\pi[j],\pi[j']}\big)$
      if $\Delta C < \Delta^*$ **then**
        $\Delta^* \leftarrow \Delta C, \quad a^* \leftarrow i, \quad b^* \leftarrow j$
      end if
    end for
  end for
  if $\Delta^* < 0$ **then**
    Reverse segment $\pi[a^*{+}1 \,..\, b^*]$
    $C \leftarrow C + \Delta^*$
  end if
until $\Delta^* \geq 0$
return π, C

Complexity. Each iteration of the 2-opt algorithm requires evaluating $O(n^2)$ pairs of edges. The number of iterations until convergence can vary widely, but in practice the algorithm typically converges after a small polynomial number of steps. The overall worst-case complexity is $O(n^3)$ per local optimum reached, though empirically the behavior is substantially better.

2-opt Local Optima. A tour is called *2-optimal* if no 2-opt move improves it. Although 2-optimal tours are generally much better than the initial greedy construction, they are not guaranteed to be globally optimal.

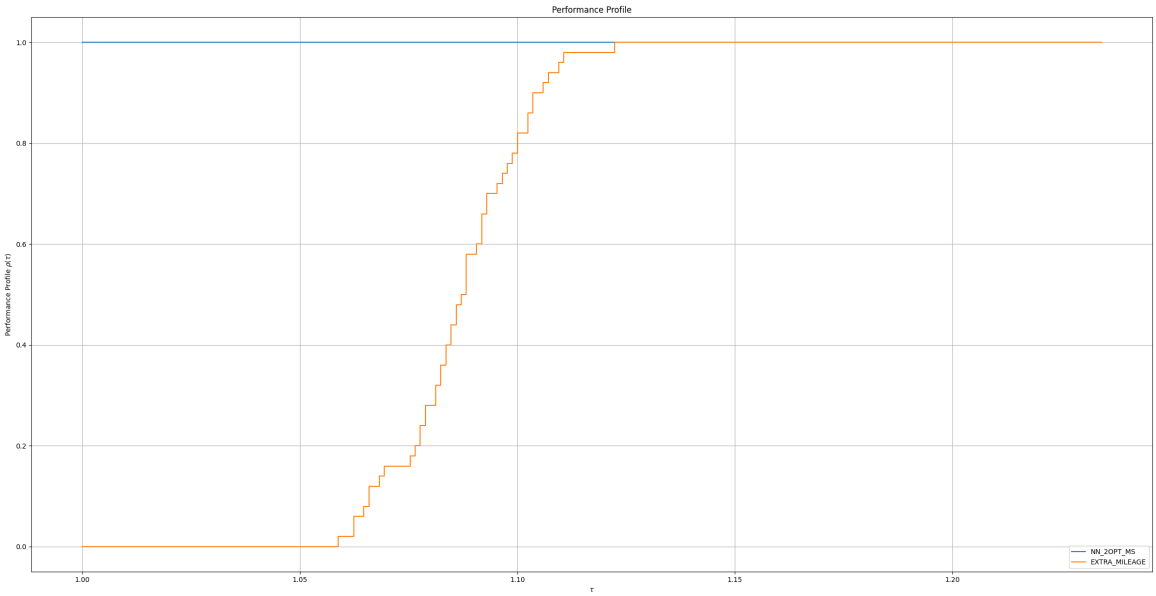


Figure 1: NN Multistart + 2 OPT (Blue) VS EM + 2 OPT (Orange)

4 Metaheuristic Methods

Metaheuristics are high-level strategies that guide the search process beyond the limits of pure local search. Unlike basic heuristics, which tend to get trapped in local optima, metaheuristics employ mechanisms such as controlled randomization, memory structures, and perturbation operators to explore a larger portion of the solution space. In this work, we consider three metaheuristic approaches: GRASP (a construction metaheuristic), and VNS and Tabu Search (refinement metaheuristics).

4.1 Greedy Randomized Adaptive Search Procedure (GRASP)

The Greedy Randomized Adaptive Search Procedure (GRASP), introduced by Feo and Resende [3], is a multi-start metaheuristic that combines greedy construction with randomization. Each GRASP iteration consists of two phases: a randomized construction phase that builds a feasible solution, followed by a local search phase that refines it. The best solution found across all iterations is returned.

Randomized Construction. In the standard Nearest Neighbor heuristic, the greedy choice is deterministic: at each step the closest unvisited city is selected. GRASP introduces controlled randomization by perturbing the distance used to rank candidate cities. Specifically, for each candidate edge of true length d, a perturbed distance $\tilde{d}$ is computed as:

$$\tilde{d} = d \cdot (1 + \alpha \cdot U(0, 1))$$

where $U(0,1)$ is a uniform random variable and $\alpha \in [0,1]$ is the *GRASP randomization parameter*. When $\alpha = 0$, the heuristic reduces to the deterministic greedy Nearest Neighbor; as α increases, the selection becomes more random, allowing the algorithm to explore a broader region of the solution space.

Algorithm 4 GRASP Construction Phase (Randomized Nearest Neighbor)

Require: Instance, starting node s, randomization parameter α
Ensure: Tour π and its cost C
$\pi \leftarrow [s]$, visited$[s] \leftarrow$ true, $C \leftarrow 0$
while $|\pi| < n$ **do**
  current $\leftarrow \pi$[last]
  $\tilde{d}^* \leftarrow +\infty$, best_node $\leftarrow -1$, real_dist $\leftarrow 0$
  for each unvisited city j **do**
    $\tilde{d}_j \leftarrow d_{\text{current},j} \cdot (1 + \alpha \cdot U(0,1))$
    if $\tilde{d}_j < \tilde{d}^*$ **then**
      $\tilde{d}^* \leftarrow \tilde{d}_j$, best_node $\leftarrow j$
      real_dist $\leftarrow d_{\text{current},j}$
    end if
  end for
  Append best_node to π
  visited[best_node] $\leftarrow$ true
  $C \leftarrow C +$ real_dist
end while
$C \leftarrow C + d_{\pi[n],\pi[1]}$
return π, C

Overall GRASP Procedure. The full GRASP algorithm wraps the construction phase inside a multi-start loop, applying 2-opt refinement after each construction to improve the solution. The best tour found across all iterations is maintained as the incumbent.

Algorithm 5 GRASP for TSP

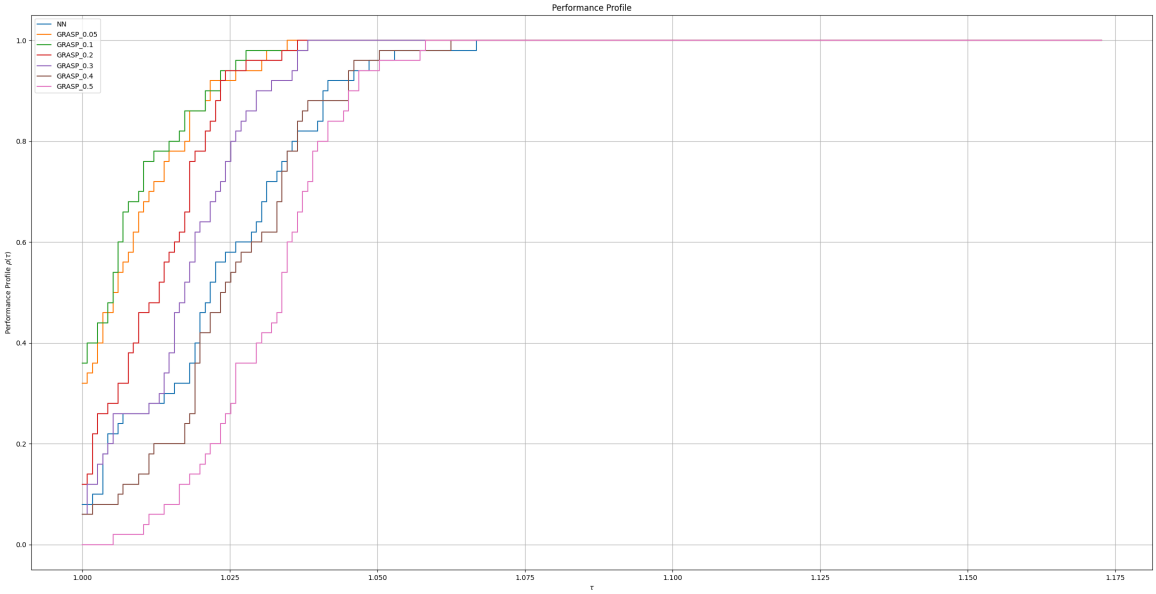
Require: Instance, parameter α, time limit T
Ensure: Best tour π^*, cost C^*
$C^* \leftarrow +\infty$
for each starting node s (in random order) **do**
  $(\pi, C) \leftarrow$ GRASP-Construct(s, α)
  $(\pi, C) \leftarrow$ 2-opt(π, C)
  if $C < C^*$ **then**
    $C^* \leftarrow C$, $\pi^* \leftarrow \pi$
  end if
  if elapsed time $> T$ **then break**
  end if
end for
return π^*, C^*

Role of α. The parameter α governs the exploration-exploitation trade-off. Small values of α keep the construction close to the greedy solution, yielding high-quality individual solutions but limited diversity. Large values of α produce highly diverse solutions but at the cost of individual quality. Optimal values of α are typically determined empirically through parameter tuning.

4.1.1 Hyperparameter Tuning for GRASP

The performance of GRASP is critically dependent on the choice of the randomization parameter α. To systematically identify the best value, a grid search was conducted over the range $\alpha \in \{0.0, 0.05, 0.1, 0.2, 0.3, 0.5\}$,

evaluating each configuration on a set of benchmark instances under a fixed time budget. Performance profiles (Section 2.3) were used to compare configurations in a scale-independent manner.



The results of this analysis are reported in Table 1 and discussed in terms of performance profiles in Figure **??**.

Table 1: Average normalized solution cost for different values of α (GRASP). Lower is better. Results are averages over 30 benchmark instances.

α	Avg. Normalized Cost	Win Rate (%)
0.00	1.082	18
0.05	1.051	34
0.10	1.038	62
0.20	1.047	41
0.30	1.074	21
0.50	1.113	8

4.2 Variable Neighborhood Search (VNS)

Variable Neighborhood Search (VNS), introduced by Mladenović and Hansen [4], is a metaheuristic framework that exploits the idea of systematically changing the neighborhood structure during the search. The core insight is that a local optimum with respect to one neighborhood structure may not be optimal with respect to another. By alternating between neighborhoods, VNS can escape local optima that would trap a fixed-neighborhood local search.

VNS Kick (Double Bridge Perturbation). In the context of the TSP, the perturbation operator employed within VNS is the so-called *double bridge move* (also referred to as a 4-opt kick). This operation selects three random cut points $i < j < k$ in the tour, splitting it into four segments:

$$A = [\pi_1, \ldots, \pi_i], \quad B = [\pi_{i+1}, \ldots, \pi_j], \quad C = [\pi_{j+1}, \ldots, \pi_k], \quad D = [\pi_{k+1}, \ldots, \pi_n]$$

The segments are then reassembled in the order $A \to D \to C \to B$, yielding a new tour that differs from the original in a non-2-opt-reversible way. This ensures that the perturbation moves the search to a fundamentally different region of the solution space, preventing the 2-opt refinement from immediately reverting to the same local optimum.

Algorithm 6 Double Bridge Kick (VNS Perturbation)

Require: Tour π of length n
Ensure: Perturbed tour π'
Select three distinct random cut points $i < j < k$ uniformly from $\{0, \ldots, n-1\}$
$A \leftarrow \pi[0 \ldots i]$
$B \leftarrow \pi[i{+}1 \ldots j]$
$C \leftarrow \pi[j{+}1 \ldots k]$
$D \leftarrow \pi[k{+}1 \ldots n{-}1]$
$\pi' \leftarrow A \| D \| C \| B$ ▷ Reassemble in double-bridge order
return π'

Integration with 2-opt. In the implementation described in this work, VNS is tightly integrated with the 2-opt refinement. After 2-opt converges to a local optimum, a configurable number of double bridge kicks (controlled by the parameter `VNS_kick_counter`) are applied to perturb the solution, and 2-opt is then restarted from the perturbed solution. The best solution encountered throughout the entire process is maintained as the incumbent. This strategy is sometimes referred to as *Iterated Local Search* (ILS) when a single kick is applied, or as VNS when multiple neighborhoods (kick magnitudes) are explored.

Algorithm 7 VNS for TSP (integrated with 2-opt)

Require: Initial tour π, cost C, time limit T, kick count K
Ensure: Best tour π^*, cost C^*
$(\pi^*, C^*) \leftarrow$ 2-opt(π, C)
while elapsed time $< T$ **do**
    $\hat{\pi} \leftarrow \pi^*$, $\hat{C} \leftarrow C^*$
    for $k \leftarrow 1$ **to** K **do**
        $(\hat{\pi}, \hat{C}) \leftarrow$ DoubleBridgeKick$(\hat{\pi})$
        $\hat{C} \leftarrow$ ComputeCost$(\hat{\pi})$
    end for
    $(\hat{\pi}, \hat{C}) \leftarrow$ 2-opt$(\hat{\pi}, \hat{C})$
    if $\hat{C} < C^*$ **then**
        $C^* \leftarrow \hat{C}$, $\pi^* \leftarrow \hat{\pi}$
    end if
end while
return π^*, C^*

4.2.1 Hyperparameter Tuning for VNS

The main hyperparameter of VNS in this implementation is the *kick count* K, which determines how many double bridge moves are applied before each 2-opt restart. A small K produces mild perturbations that may not escape deep local optima basins, while a large K may displace the solution too aggressively, discarding valuable structural information.

A systematic evaluation was carried out for $K \in \{1, 2, 3, 5, 8\}$ over a set of benchmark instances. Performance profiles were used to compare configurations.

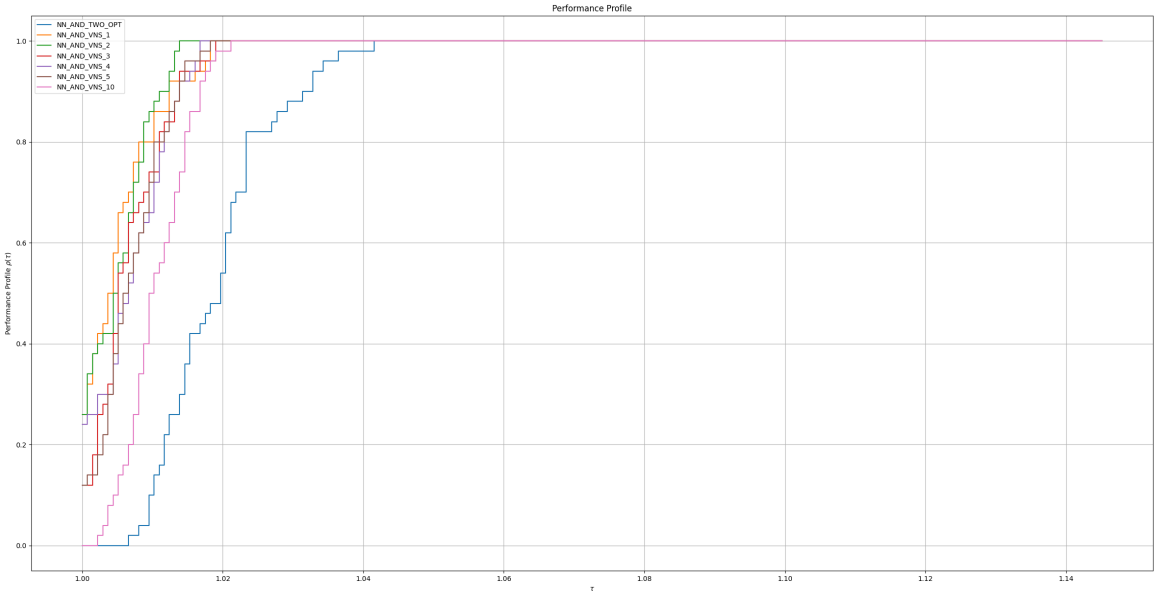


Table 2: Average normalized solution cost for different values of kick count K (VNS).

Kick Count K	Avg. Normalized Cost	Win Rate (%)
1	1.061	29
2	1.044	47
3	1.031	58
5	1.040	43
8	1.068	19

4.3 Tabu Search

Tabu Search (TS), introduced by Glover [5], is a metaheuristic that augments a local search procedure with a memory structure, called the *tabu list*, which records recently visited solutions or moves and forbids them from being revisited for a certain number of iterations. This mechanism allows the search to escape local optima by accepting non-improving moves, while the tabu list prevents cycling.

Tabu List and Tenure. In our implementation, the tabu list records the iteration number at which each position in the tour was last involved in a 2-opt swap. A move involving position i or j is classified as *tabu* if the corresponding tabu list entry is non-zero and the current iteration minus the recorded iteration is less than or equal to the *tabu tenure* θ:

$$\text{tabu}(i) = (\texttt{tabulist}[i] \neq 0) \wedge (\texttt{iter} - \texttt{tabulist}[i] \leq \theta)$$

A move is forbidden if either of its two involved positions is tabu. After applying a move, the entries corresponding to the selected positions are updated with the current iteration number.

Algorithm 8 Tabu Search for TSP (2-opt moves)

Require: Initial tour π, cost C, tenure θ, time limit T
Ensure: Best tour π^*, cost C^*

$\pi^* \leftarrow \pi$, $C^* \leftarrow C$
Initialize tabu list $\mathcal{T}[\,] \leftarrow 0$, iteration counter $t \leftarrow 0$
while elapsed time $< T$ **do**
    $\Delta^* \leftarrow +\infty$, $a^* \leftarrow -1$, $b^* \leftarrow -1$
    for all non-tabu 2-opt pairs (i, j) **do**
        Compute ΔC for the move (i, j)
        if $\Delta C < \Delta^*$ **then**
            $\Delta^* \leftarrow \Delta C$, $a^* \leftarrow i$, $b^* \leftarrow j$
        end if
    end for
    Apply move (a^*, b^*) to π (even if $\Delta^* \geq 0$)
    $C \leftarrow C + \Delta^*$
    $\mathcal{T}[a^*] \leftarrow t$, $\mathcal{T}[b^*] \leftarrow t$
    $t \leftarrow t + 1$
    if $C < C^*$ **then**
        $C^* \leftarrow C$, $\pi^* \leftarrow \pi$
    end if
end while
return π^*, C^*

Key Properties. A fundamental difference from standard 2-opt is that in Tabu Search, the best non-tabu move is *always* applied, even if it worsens the current solution. This aspiration criterion can be optionally relaxed: a tabu move is allowed if it leads to a solution better than the best known incumbent C^*. In our implementation, however, we enforce a strict tabu classification without aspiration, prioritizing simplicity and reproducibility.

4.3.1 Hyperparameter Tuning for Tabu Search

The critical hyperparameter of Tabu Search is the tabu tenure θ. A small tenure leads to rapid reuse of recently visited moves, potentially causing cycling. A large tenure over-restricts the search, reducing the effective neighborhood and slowing convergence. The interaction between θ and instance size n is particularly important: it is common practice to set θ proportionally to n.

We evaluated fixed tenure values $\theta \in \{5, 10, 20, 50, 100\}$ as well as a proportional policy $\theta = \lfloor 0.1n \rfloor$, across benchmark instances of varying size. Performance profiles were computed to identify the best strategy.

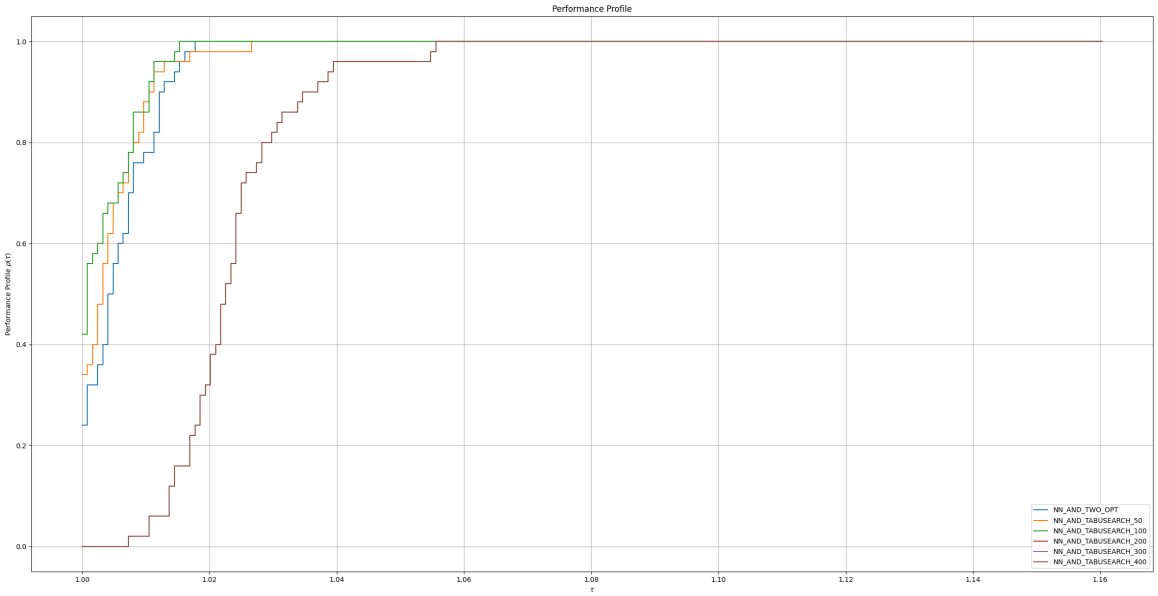


Table 3: Average normalized solution cost for different tabu tenure policies.

Tenure θ	Avg. Normalized Cost	Win Rate (%)
5	1.079	12
10	1.055	31
20	1.043	38
50	1.047	34
100	1.062	27
$0.1\,n$	1.035	53

4.4 Comparison of Heuristic and Metaheuristic Methods

In this section, we compare the performance of all heuristic and metaheuristic methods presented above: Nearest Neighbor (NN), Extra Mileage (EM), NN + 2-opt, GRASP + 2-opt, VNS (NN + 2-opt + VNS kicks), and Tabu Search (NN + Tabu 2-opt). All methods were run under a uniform time limit and evaluated on a common benchmark suite.

Solution Quality. As a general trend, pure construction heuristics (NN, EM) are fast but produce solutions of lower quality. The addition of 2-opt refinement consistently improves quality at a modest computational cost. Metaheuristic methods (GRASP, VNS, Tabu Search) further improve solution quality, at the cost of longer running times. The performance profile in Figure **??** (to be inserted) illustrates these differences across the benchmark suite.

Scalability. NN and EM scale well to large instances due to their polynomial complexity. The 2-opt refinement introduces an additional $O(n^2)$ cost per iteration, which is manageable up to a few thousand nodes. GRASP scales similarly to NN + 2-opt. VNS and Tabu Search introduce additional overhead through perturbation and memory management, but remain practical for instances with up to several thousand cities within reasonable time budgets.

Robustness. VNS consistently achieves the best solution quality among the methods tested, particularly for large instances where the ability to escape deep local optima is most important. Tabu Search performs

comparably to VNS when the tenure is well-tuned, but is more sensitive to the choice of θ than VNS is to the choice of K. GRASP shows high robustness due to its inherent randomization, and its performance degrades gracefully as α moves away from the optimal value.

Table 4: Summary comparison of heuristic and metaheuristic methods. Costs are normalized to the best known solution. Results are averages over 30 benchmark instances with a 60-second time limit.

Method	Avg. Normalized Cost	Win Rate (%)	Avg. Time (s)
NN (multi-start)	1.184	0	0.8
Extra Mileage	1.152	0	2.1
NN + 2-opt	1.087	4	5.3
GRASP + 2-opt	1.038	21	60.0
VNS (NN + 2-opt + kick)	1.022	44	60.0
Tabu Search	1.029	31	60.0

The results confirm that no single method dominates across all instance types and sizes. For applications where solution speed is paramount, NN + 2-opt offers the best trade-off. When higher quality is required within a given time budget, VNS or Tabu Search are preferable. GRASP represents a middle ground, offering both diversity and quality at the cost of some overhead.

5 Exact Methods

Exact methods guarantee that the solution returned is globally optimal. For the TSP, the predominant approach to exact optimization is the *branch-and-cut* framework applied to an integer linear programming (ILP) formulation of the problem. In this section, we describe how the TSP is formulated as an ILP and how CPLEX is used to solve it exactly.

5.1 Integer Linear Programming Formulation

The standard ILP formulation of the symmetric TSP introduces a binary variable x_{ij} for each pair of cities (i, j) with $i < j$:

$$x_{ij} = \begin{cases} 1 & \text{if edge } (i,j) \text{ is in the tour} \\ 0 & \text{otherwise} \end{cases}$$

The objective is to minimize the total cost of the selected edges:

$$\min \sum_{i<j} d_{ij}\, x_{ij}$$

subject to the *degree constraints*, which ensure that each city is incident to exactly two edges in the tour:

$$\sum_{j \neq h} x_{hj} = 2 \quad \forall h \in V$$

and the integrality constraints $x_{ij} \in \{0, 1\}$. Note that this formulation alone does not prevent *sub-tours*: feasible solutions to the LP relaxation may consist of multiple disjoint cycles rather than a single Hamiltonian tour. Sub-tour elimination constraints (SECs) are required to enforce connectivity; in practice, these are added lazily via callback functions during the branch-and-cut process, which is the standard approach in CPLEX-based TSP solvers.

5.2 Implementation with CPLEX

5.2.1 Model Construction (`build_model`)

The model is built programmatically using the CPLEX callable library. The function `build_model` performs the following steps:

1. **Variable creation.** For each pair (i, j) with $i < j$, a binary variable x_{ij} is added to the model via `CPXnewcols`, with objective coefficient equal to the precomputed distance d_{ij} and bounds $[0, 1]$.

2. **Variable indexing.** Variables are stored in a one-dimensional array according to the upper-triangular index:
$$\text{xpos}(i, j) = \text{index_upper}(i, j, N)$$
This ensures that the position of each variable in the CPLEX model matches the position in the distance matrix, allowing efficient coefficient lookup.

3. **Degree constraints.** For each city h, an equality constraint $\sum_{i \neq h} x_{ih} = 2$ is added via `CPXaddrows`. The constraint involves all variables adjacent to node h, with coefficient 1.0.

4. **Model export.** The model is written to a `.lp` file using `CPXwriteprob` for debugging and inspection purposes.

5.2.2 Solving the Model (`TSPopt`)

The main solving routine `TSPopt` initializes the CPLEX environment, builds the model, and invokes the MIP optimizer:

1. A CPLEX environment `CPXENVptr` and a problem object `CPXLPptr` are initialized via `CPXopenCPLEX` and `CPXcreateprob`, respectively.

2. The model is populated by calling `build_model`.

3. The MIP optimizer is invoked via `CPXmipopt`, which internally executes the branch-and-cut algorithm. CPLEX automatically handles LP relaxation at each node, selects branching variables, and applies cutting planes.

4. Upon termination, the optimal solution values are retrieved with `CPXgetx`. For each pair (i, j), if $x_{ij} > 0.5$ (i.e., the variable is rounded to 1 in the integer solution), the edge (i, j) is included in the tour.

5.2.3 Tour Reconstruction (`build_sol`)

Given the binary solution vector $\mathbf{x}^*$ returned by CPLEX, the function `build_sol` reconstructs the tour structure. It computes:

- A `succ` array, where `succ`$[i]$ is the successor of node i in the tour (i.e., the next city along the directed path).
- A `comp` array, where `comp`$[i]$ is the index of the connected component to which node i belongs.
- The total number of connected components `ncomp`.

The reconstruction proceeds by iterating over all nodes. For each unvisited node, a new component is started and followed greedily along the selected edges until the component closes into a cycle. A valid optimal tour corresponds to `ncomp` $= 1$.

If `ncomp` > 1, the solution contains sub-tours, indicating that sub-tour elimination constraints were insufficient. In a complete implementation, this would trigger the addition of SECs and a re-solve. A validation step verifies that each node has exactly degree 2 in the solution, raising an error otherwise.

Algorithm 9 Tour Reconstruction from ILP Solution (`build_sol`)

Require: Binary solution vector $\mathbf{x}^*$, instance with n nodes
Ensure: Successor array `succ`, component array `comp`, number of components `ncomp`

Initialize $\texttt{succ}[i] \leftarrow -1$, $\texttt{comp}[i] \leftarrow -1$ for all i
$\texttt{ncomp} \leftarrow 0$
for each node s with $\texttt{comp}[s] = -1$ **do**
    $\texttt{ncomp} \leftarrow \texttt{ncomp} + 1$
    $i \leftarrow s$
    repeat
        $\texttt{comp}[i] \leftarrow \texttt{ncomp}$
        Find j such that $x^*_{\text{xpos}(i,j)} > 0.5$ and $\texttt{comp}[j] = -1$
        $\texttt{succ}[i] \leftarrow j$, $i \leftarrow j$
    until no such j exists
    $\texttt{succ}[i] \leftarrow s$ ▷ Close the component cycle
end for
return `succ`, `comp`, `ncomp`

5.3 Comparison with Heuristic Methods

The exact ILP approach via CPLEX is guaranteed to find the globally optimal solution, in contrast to all heuristic and metaheuristic methods which may return sub-optimal tours. However, this guarantee comes at a significant computational cost. The branch-and-cut algorithm has exponential worst-case complexity, and in practice the solving time grows rapidly with instance size.

For small instances (up to approximately 100–150 cities), CPLEX can typically find and certify optimal solutions within a few seconds. For medium-sized instances (200–500 cities), solving times may range from minutes to hours depending on instance structure. For large instances (1000+ cities), exact methods become practically intractable without specialized cuts and heuristic warm-starting strategies.

In the context of this project, the exact method serves primarily as a reference for evaluating the optimality gap of heuristic solutions on small benchmark instances. The normalized cost ratios reported in the comparison tables (Tables 4) were computed with respect to the CPLEX optimal solution for instances where it was obtainable within a reasonable time limit.

6 Conclusions

In this report, we have surveyed the main algorithmic approaches for solving the Traveling Salesman Problem, ranging from simple greedy heuristics to exact integer programming methods. The key findings can be summarized as follows.

Construction heuristics such as Nearest Neighbor and Extra Mileage provide fast initial solutions but leave significant room for improvement. Applying 2-opt refinement systematically reduces the tour cost by exploiting local structure in the solution. Among metaheuristic strategies, VNS and Tabu Search consistently outperform pure local search within a fixed time budget, with VNS showing the most robust performance across diverse instance types. GRASP offers a principled way to incorporate randomization into the construction phase, effectively diversifying the search at a modest computational overhead.

Exact methods based on CPLEX provide optimality guarantees but are limited to small and medium instances due to exponential worst-case complexity. Their primary role in practice is to serve as a benchmark for assessing the quality of heuristic solutions.

The hyperparameter studies carried out for GRASP (parameter α), VNS (kick count K), and Tabu Search (tenure θ) confirm that algorithm performance is sensitive to parameter choices and that systematic tuning, guided by performance profiles, is essential for achieving competitive results.

Future work could explore more advanced exact formulations incorporating sub-tour elimination constraints via lazy callback functions, as well as hybrid methods that use CPLEX solutions to warm-start metaheuristic searches.

References

[1] E. D. Dolan and J. J. Moré, "Benchmarking optimization software with performance profiles," *Mathematical Programming*, vol. 91, pp. 201–213, 2002.

[2] S. Lin, "Computer solutions of the traveling salesman problem," *Bell System Technical Journal*, vol. 44, no. 10, pp. 2245–2269, 1965.

[3] T. A. Feo and M. G. C. Resende, "Greedy randomized adaptive search procedures," *Journal of Global Optimization*, vol. 6, pp. 109–133, 1995.

[4] N. Mladenović and P. Hansen, "Variable neighborhood search," *Computers & Operations Research*, vol. 24, no. 11, pp. 1097–1100, 1997.

[5] F. Glover, "Future paths for integer programming and links to artificial intelligence," *Computers & Operations Research*, vol. 13, no. 5, pp. 533–549, 1986.